

Semnalizarea handshake pentru transferuri de date

Probleme propuse

Oprîtoiu Flavius
flavius.opritoiu@cs.upt.ro

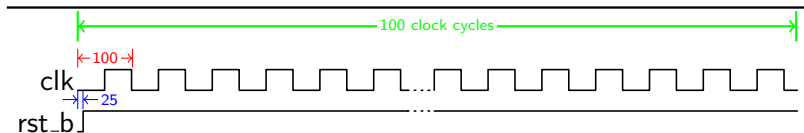
28 septembrie 2026

Problema 1

Construiți un modul numit *prod* (producător) având intrările *clk* (1-bit) și *rst_b* (1-bit) și ieșirile *val* (1-bit) și *data* (8-biți). Modulul produce valori aleatorii între 0 și 5. Ieșirea *val* (semnal handshaking *valid*) anunță când unitatea nu poate produce o nouă valoare (*val* == 0).

Modulul generează date valide cel puțin 3 și cel mult 5 cicluri de tact după care va aștepta (fără să genereze date) cel puțin 1 și cel mult 4 cicluri de tact. Construiți:

1. codul Verilog
2. fișierul script, "run_prod.txt"
3. un testbench care generează intrările ca în diagrama de timp de mai jos

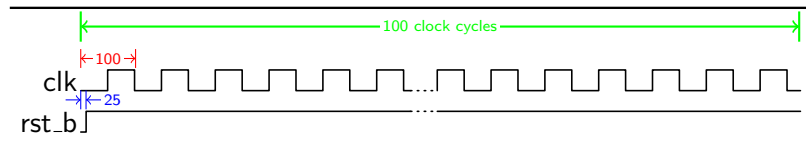


Problema 2

Construiți un consumator numit *cons* (consumator) având intrările *clk* (1-bit), *rst_b* (1-bit), *val* (1-bit) și *data* (8-biți) și având o ieșire *sum* (8-biți). Unitatea consumă date generate de producător. La ieșirea *sum*, consumatorul furnizează suma tuturor datelor valide recepționate. Testbench-ul va instanția producătorul de la Exercise 1 și va conecta cele 2 module.

Construiți:

1. codul Verilog
2. fișierul script, "run_cons.txt"
3. un testbench care generează intrările ca în diagrama de timp de mai jos



Problema 3

Modificați modulul consumator din Problema 2 astfel încât, în orice moment, la ieșirea lui să furnizeze suma celor mai recente secvențe monotonic crescătoare de date valide.

O dată validă face parte dintr-o secvență monoton crescătoare dacă este mai mare sau egală ca data validă precedentă. Exemplu: secvența 0, 1, 1, 3, 3, 3, 5 este monoton crescătoare, iar pentru această secvență, ieșirea consumatorului este așteptată să fie 0, 1, 2, 5, 8, 11, 16. Secvența 0, 1, 4, 3, 2, 3, 5 este alcătuită din 3 subsecvențe monotone crescătoare, iar ieșirea consumatorului este așteptată să fie 0, 1, 5, 3, 2, 5, 10.

Problema 4

Construiți unitatea de control descrisă în Problema rezolvată a materialului din această săptămână, numită *sha2inctrl*.

Testbench-ul va instanția modulul *sha2indpath* construit săptămâna trecută, care va fi controlat de această unitate de control și va conecta cele 2 module.

Construiți:

1. codul Verilog
2. fișierul script, "run_sha2inctrl.txt"
3. un testbench care generează intrările ca în diagrama de timp de mai jos

